

SIL765: Network and System Security

Assignment 1: Basic Cryptanalysis

Nizaul Rahman

Entry Number: 2025MCS2099

Submitted: January 25, 2026

1 Overview

This report documents the methodology and results for Assignment 1, which required the cryptanalysis of a Monoalphabetic Substitution Cipher. The objective was to recover the plaintext and the secret key from given ciphertexts without prior knowledge of the mapping. The solution implemented is generic and capable of deciphering unseen texts by utilizing statistical properties of the English language.

2 Methodology

To break the substitution cipher, I implemented a **Stochastic Hill Climbing Algorithm** combined with **Quadgram Frequency Analysis**. This approach treats cryptanalysis as an optimization problem where the goal is to maximize the “fitness” (English-likeness) of the deciphered text.

2.1 The Scoring Function (Fitness)

The core of the solver relies on N -gram statistics. I utilized a standard reference file of English Quadgrams (4-letter sequences like TION, THEY). The fitness of a candidate decryption is calculated as the sum of the log-probabilities of every quadgram found in the text:

$$Score(T) = \sum_{i=0}^{L-4} \log_{10}(P(q_i)) \quad (1)$$

Where:

- T is the candidate deciphered text.
- q_i is the quadgram at position i .
- $P(q_i)$ is the probability of that quadgram appearing in standard English.

Logarithms are used to prevent arithmetic underflow, as multiplying many small probabilities results in numbers too small for standard floating-point representation.

2.2 The Algorithm

The solver operates in the following steps:

1. **Initialization:** Generate a random key map (randomly associating ciphertext characters to plaintext 'a'-'z').
2. **Evaluation:** Decipher the text using the current key and compute its Score.
3. **Optimization (The Climb):**
 - Swap two characters in the key map randomly.
 - Re-decipher and re-score the text.
 - If the new score is higher, keep the swap (move up the hill).
 - If the new score is lower, revert the swap.
4. **Random Restarts:** The Hill Climbing algorithm can get stuck in local maxima (sub-optimal solutions). To ensure the global maximum (the correct English text) is found, the algorithm automatically restarts with a fresh random key 20–100 times. The result with the highest overall score is selected as the final answer.

3 Execution Instructions

The solution is implemented in Python and includes a bash script for automated testing against 6 test cases.

- **Prerequisites:** Python 3.x and the `english_quadgrams.txt` file must be present in the directory.

- **Running a single case:**

```
python3 solver.py < inputs/input-1.txt
```

- **Running the test suite:**

```
./checkscript.sh
```

4 Deciphered Results

Below are the recovered plaintexts and the corresponding key mappings for the three provided ciphertexts.

4.1 Ciphertext 1

Deciphered Plaintext:

india, officially the republic of india, is a country in south asia. it is the seventh largest country by area, the second most populous country, and the most populous democracy in the world. bounded by the indian ocean on the south, the arabian sea on the southwest, and the bay of bengal on the southeast, it shares land borders with pakistan to the west; china, nepal, and bhutan to the north; and bangladesh and myanmar to the east. in the indian ocean, india is in the vicinity of sri lanka and the maldives; its andaman and nicobar islands share a maritime border with thailand, myanmar and indonesia. good, now turn for the second part of the question, good luck!

Key Mapping:

Key String: y5n8@p7q1xwu09\$342vos6#xxx

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
y	5	n	8	@	p	7	q	1	x	w	u	0	9	\$	3	4	2	v	o	s	6	#	x	x	x

Table 1: Mapping for Ciphertext 1

4.2 Ciphertext 2**Deciphered Plaintext:**

defeated and leaving his dinner untouched, he went to bed. that night he did not sleep well, having feverish dreams, having no rest. he was unsure whether he was asleep or dreaming. conscious, unconscious, all was a blur. he remembered crying, wishing, hoping, begging, even laughing. he floated through the universe, seeing stars, planets, seeing earth, all but himself. when he looked down, trying to see his body, there was nothing. it was just that he was there, but he could not feel anything for just his presence.

Key Mapping:

Key String: 8ot64spnrzxqwy\$1x3vu205x#x

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
8	o	t	6	4	s	p	n	r	x	z	q	w	y	\$	1	x	3	v	u	2	0	5	x	#	x

Table 2: Mapping for Ciphertext 2

4.3 Ciphertext 3**Deciphered Plaintext:**

bhagat singh was a charismatic indian revolutionary who participated in the murder of a junior british police officer and an indian head constable in mistaken retaliation for the death of an indian nationalist. he also took part in a largely symbolic bombing of the central legislative assembly in delhi and a hunger strike in jail, which on the back of sympathetic coverage in indian owned newspapers turned him into a household name in punjab region, and after his execution caused by the british rulers at age twenty three into a martyr and folk hero in northern india.

Key Mapping:

Key String: 648otsp73#9rxzqwx\$n1vu205x

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
6	4	8	o	t	s	p	7	3	#	9	r	x	z	q	w	x	\$	n	1	v	u	2	0	5	x

Table 3: Mapping for Ciphertext 3

5 Project Directory Structure

The submitted project folder follows a structured hierarchy to separate source code, data, and testing artifacts. Below is the description of the folders and files present in the submission:

solver.py The main Python executable that implements the Stochastic Hill Climbing algorithm. It reads ciphertext from standard input and prints the deciphered plaintext and key.

english_quadgrams.txt The statistical dataset containing the log-probabilities of English quadgrams (4-letter sequences). This is required by **solver.py** for the fitness scoring function.

checkscript.sh The automated testing script provided for the assignment. It runs the solver against the input files and verifies the output against the ground-truth answers.

inputs/ A directory containing the ciphertext files used as test cases.

- **input-1.txt** to **input-3.txt**: The original assignment ciphertexts.
- **input-4.txt** to **input-6.txt**: Additional generated ciphertexts to test the generality of the solver.

answers/ A directory containing the ground-truth text files. The checkscript compares the solver's output against these files to determine correctness.

outputs/ A directory generated during execution. The solver writes its results (deciphered text and key) here for each test case.

org_ciphers/ A utility folder containing the original source files and helper scripts used during development:

- **gen_cases.py**: A Python script used to generate the extra test cases (4, 5, and 6) to ensure the solver is not overfitting.
- **debug_check.py**: A script used for detailed diff-checking during the debugging phase.

6 Conclusion

The implemented tool successfully deciphered all provided test cases. By using frequency analysis and randomization (restarts), the solver avoids local maxima and reliably converges to the correct substitution key, demonstrating that simple monoalphabetic substitution ciphers provide insufficient security against statistical attacks.